

Metody Programowania

Zadania do treningu

Data aktualizacji: 2026-06-18

Tablica zawartości

I	Przedmowa	3
I.1	Format zawartości	3
II	Wiązanie zmiennych	4
III	Typowanie wyrażeń	6
IV	Rekurencja ogonowa	9
V	Identyfikacja funkcji	13
VI	Funkcje na listach	15
VII	Gramatyki	19
VIII	Abstrakcyjne typy danych	29
VIII.1	Rozwinięcie typów języka	29
VIII.1.1	Język arytmetycznych wyrażeń	29
VIII.1.2	Język calc_let	30
VIII.2	Monady	31
IX	Indukcja strukturalna	33
IX.1	Formułowanie zasady indukcji	33
IX.1.1	Zadanie A.	33
IX.1.2	Zadanie B.	35

I. *Przedmowa*

Materiału do przedmiotu *Metod Programowania* nie da się nauczyć w tydzień. Nawet dwa tygodnie nie wystarczą. Najambitniejsi być może zdołają to uczynić w 2 tygodnie i 42 sekundy. Przedmiot ten wymaga systematycznej pracy, programowania w OCamlu na co dzień, realizowania zadań z list i pracowni, zadawania pytań. Zainteresowanie tematem języków programowania może usprawnić systematyczną pracę, pozwolić głębiej zrozumieć problemy zadawane na zajęciach.

Innymi słowy – jeżeli za $<$ tydzień masz egzamin i nie wiesz o czym jest ten przedmiot, to lepiej zmienić studia lub przygotowywać się do powtarzania MP. **Natomiast**, jeżeli skrupulatnie się uczyłeś, chodziłeś na wykłady i na nich uważałeś, próbowałeś podchodzić do większości zadań z list – nie masz raczej czego się bać. Ten zestaw pytań to będzie dla Ciebie jedynie powtórka z materiału całego wykładu, usystematyzowanie wiedzy, ćwiczenia mentalne. Na spokojnie można to przejrzeć dzień przed egzaminem, spróbować porozwijać parę zadań samodzielnie, a resztę dnia odpoczywać.

Jeśli jednak masz problemy z $>$ połową zadań, to na naukę zapewne za późno, ale warto próbować. Powodzenia!

I.1 *Format zawartości*

Każde zadanie jest wydrukowane na osobnej kartce, dając miejsce na samodzielne rozwiązanie/nie spojlerując odpowiedzi. Na stronach kolejnych są wszelkie wskazówki do zadania, które mogą lekko pomóc w jego rozwiązaniu. Wreszcie – rozwiązanie z wyjaśnieniem.

Ponownie, na naukę widząc ten materiał parę dni przed sesją, pewnie już za późno. Tutaj zakładam że większość rzeczy jest znane, tłumacząc tylko rzeczy mniej widoczne dla niewprawionego oka.

II. Wiązanie zmiennych

W poniższych wyrażeniach podkreśl wolne wystąpienia zmiennych. Dla każdego związanego wystąpienia zmiennej, narysuj strzałkę od tego wystąpienia do wystąpienia wiążącego je.

```
let f x =  
  
    let x = x  
  
    and y = x * y in  
  
f x y z
```

```
fun f x y ->  
  
    let z = x + y in  
  
    let x = y + x in  
  
fun y -> g x y z
```

```
let rec f x =  
  
    let rec g z y = j y (f z)  
  
    and j y z = g z (f y)  
  
in  
  
if g y > f x then  
  
    g x  
  
else  
  
    g y
```

```
let fun x = x * x in  
  
(fun x -> x * 5) y  
  
|> (fun x -> y)  
  
|> x
```

```
let zagadka b c f =  
  
    match b with  
  
    | [] -> []  
  
    | a :: b ->  
  
        zagadka b (f a c) f
```

```
let z = function  
  
    | [] -> 0  
  
    | _ :: d -> 1 + (z d)  
  
and f = z  
  
and g xs ys =  
  
    (f xs) * (f ys)
```

Wskazówki do analizy zasięgu zmiennych:

- **Symultaniczne wiązanie (and):** W konstrukcji `let x = e1 and y = e2 in e3`, wyrażenia `e1` i `e2` są ewaluowane w tym samym, **zewnętrznym** środowisku (nie widzą siebie nawzajem). Zmienne `x` i `y` są widoczne dopiero w ciele `e3`.
- **Przesłanianie (shadowing):** Najbardziej wewnętrzne wiązanie danej nazwy zawsze wygrywa. Pamiętaj, że nowe wiązania wprowadzają nie tylko konstrukcje `let`, ale też argumenty funkcji (`fun x -> ...`) oraz dopasowania do wzorca (np. `| a :: b -> ...`).
- **Zasięg a rekurencja:** Jeśli konstrukcja to zwykłe `let` (bez `rec`), nazwa definiowanej funkcji/zmiennej nie jest widoczna w jej własnym ciele. Wszelkie odwołania do niej będą szukać definicji w środowisku zewnętrznym.

III. Typowanie wyrażeń

Dla poniższych wyrażeń w języku OCaml podaj ich (najogólniejszy) typ, lub napisz BRAK TYPU, gdy wyrażenie nie posiada typu (nie typuje się)

Lp	Wyrażenie	Typ
1.	<code>fun x -> x</code>	
2.	<code>fun x -> x *. 2</code>	
3.	<code>fun x y -> !x > (y+2,10)</code>	
4.	<code>let rec add a b = if b = 0 then a else add (a + 1) (b-1)</code>	
5.	<code>fun f x -> f (f x)</code>	
6.	<code>fun f -> (fun x -> f x x)(fun y -> f y y)</code>	
7.	<code>fun f x -> f (f x) > x</code>	
8.	<code>fun a b c f d g e -> f b d c (g a)</code>	
9.	<code>List.fold_right</code>	
10.	<code>List.map</code>	
11.	<code>List.iter2</code>	
12.	<code>fun f x y -> f</code>	

Wskazówki do typowania w OCamlu

1. **Brak niejawnego rzutowania typów:** OCaml jest niezwykle rygorystyczny i nigdy nie zamienia automatycznie typu `int` na `float`.
2. **Problem nieskończonego typu (Occurs check):** Zmienna typowa nie może zawierać samej siebie. Jeśli przy rozwiązywaniu równań wyjdzie, że np. funkcja wymaga, aby typ τ był jednocześnie typem $\tau \rightarrow \alpha$ (co zdarza się np. przy aplikowaniu zmiennej do samej siebie, jak w `x x`), oznacza to, że w standardowym OCamlu takie wyrażenie po prostu **nie typuje się**.
3. **Nieużywane argumenty:** Jeśli funkcja przyjmuje argument, ale w ogóle go nie wykorzystuje w swoim ciele, jego typ nie jest niczym ograniczony. Taki argument przyjmuje najbardziej ogólną postać – osobną zmienną typową (np. `'a`, `'b`, itd.).
4. **Wymuszanie struktury przez operatory:** Operatory narzucają konkretne wymagania na swoje argumenty. Operator `!` natychmiast wymusza, by jego argument był referencją (`'a ref`). Z kolei operatory porównania (np. `>`, `=`) wymagają, aby typy po obu ich stronach były **identyczne**. Jeśli z jednej strony znajduje się krotka o konkretnych typach, to wyrażenie z drugiej strony musi odpowiadać tej samej krotce.
5. **Efekty uboczne a typ zwracany:** Pamiętaj o różnicy między funkcjami transformującymi (rodzina `map`, `fold`), a iterującymi (rodzina `iter`). Funkcje zdefiniowane wyłącznie dla wywołania efektów ubocznych (jak wypisywanie na ekran) zwracają na końcu typ pusty, czyli `unit`.
6. Jak zapamiętać `List.fold_left` i `List.fold_right`? Skojarz `left` i `right` z tym, po której stronie stoi akumulator w funkcji transformującej oraz kolejnych argumentach:

- `List.fold_left`:

Akumulator stoi po lewej stronie w lambdzie i argumentach po niej

```
('acc -> 'a -> 'acc) -> 'acc -> 'a list -> 'acc
```

- `List.fold_right`:

Akumulator stoi po prawej stronie w lambdzie i argumentach po niej

```
('a -> 'acc -> 'acc) -> 'a list -> 'acc -> 'acc
```

Rozwiązania

Lp	Wyrażenie	Typ
1.	<code>fun x -> x</code>	<code>'a -> 'a</code>
2.	<code>fun x -> x *. 2</code>	BŁĄD TYPU – 2 nie jest typu float
3.	<code>fun x y -> !x > (y+2,10)</code>	<code>(int * int) ref -> int -> bool</code>
4.	<code>let rec add a b = if b = 0 then a else add (a + 1) (b-1)</code>	<code>int -> int -> int</code>
5.	<code>fun f x -> f (f x)</code>	<code>('a -> 'a) -> 'a -> 'a</code>
6.	<code>fun f -> (fun x -> f x x)(fun y -> f y y)</code>	BRAK TYPU – 'a występuje w 'a -> 'b
7.	<code>fun f x -> f (f x) > x</code>	<code>('a -> 'a) -> 'a -> bool</code>
8.	<code>fun a b c f d g e -> f b d c (g a)</code>	<code>'a -> 'b -> 'c -> ('b -> 'd -> 'c -> 'g -> 'f) -> 'd -> ('a -> 'g) -> 'e -> 'f</code>
9.	<code>List.fold_right</code>	<code>('a -> 'acc -> 'acc) -> 'a list -> 'acc -> 'acc</code>
10.	<code>List.map</code>	<code>('a -> 'b) -> 'a list -> 'b list</code>
11.	<code>List.iter2</code>	<code>('a -> 'b -> unit) -> 'a list -> 'b list -> unit</code>
12.	<code>fun f x y -> f</code>	<code>'a -> 'b -> 'c -> 'a</code>

IV. Rekurencja ogonowa

Określ czy definicje poniższych funkcji rekurencyjnych są ogonowe. Jeżeli tak, wpisz w komórce obok TAK, jeżeli nie – przepisz je na ogonowe, lub uzasadnij, dlaczego to niemożliwe.

Lp	Definicja funkcji	Odpowiedź
1.	<pre>let rec f a = if a < 0 then -1 else if a = 0 then a else a + f (a-1)</pre>	
2.	<pre>let rec comp f fil a = if fil a then a else comp f fil (f a)</pre>	
3.	<pre>let rec f g xs = match xs with [] -> [g 0] x :: xs -> (g x) :: (f g xs)</pre>	
4.	<pre>let rec app xs ys = match xs with [] -> ys x :: xs -> x :: (app xs ys)</pre>	
5.	<pre>let rec comp f fil a = if fil a then a else 1 + comp f fil (f a)</pre>	
6.	<pre>let rec f g xs acc = match xs with [] -> acc x :: xs -> f g xs (g acc x)</pre>	

Wskazówki do rekurencji ogonowej

Funkcję nazywamy **rekurencyjnie ogonową**, **ogonową**, **tail call** wtedy, gdy wywołanie rekurencyjne jest zwracane bezpośrednio jako wynik, nie robi nic z wynikiem wywołania rekurencyjnego. To pozwala na sporą optymalizację kompilatora, gdyż nie musi wtedy zapisywać i zwiększać stosu wywołań.

Funkcję nieogonową zazwyczaj da się przekształcić na ogonową przez dodanie akumulatora który gromadzi aktualny stan wykonania. Należy jednak mieć na uwadze, że akumulator jest wtedy komputowany w odwrotnej kolejności – od pierwszego wywołania do ostatniego, w przeciwieństwie do stosu wywołań i obliczeń na wyniku wywołania funkcji rekurencyjnej.

Lp	Definicja funkcji	Odpowiedź
1.	<pre>let rec f a = if a < 0 then -1 else if a = 0 then a else a + f (a-1)</pre>	<pre>let f a = let rec aux a acc = if a < 0 then acc - 1 (* acc będzie zawsze 0 *) else if a = 0 then a + acc else aux (a-1) (acc + a) in aux a 0</pre>
2.	<pre>let rec comp f fil a = if fil a then a else comp f fil (f a)</pre>	TAK
3.	<pre>let rec f g xs = match xs with [] -> [g 0] x :: xs -> (g x) :: (f g xs)</pre>	Ta funkcja nie jest ogonowa. W klasycznym, czysto funkcyjnym ujęciu nie da się jej przekształcić w funkcję ogonową bez ponoszenia dodatkowych kosztów wydajnościowych. @, List.fold_right same nie są ogonowe (wg. domyślnej implementacji OCaml'a). Najczęstszą metodą stosowaną w takich sytuacjach, jeśli naprawdę potrzebujemy ogonowej funkcji ze względu na olbrzymią długość przekazywanej listy jest tzw. two-pass. Najpierw odwracamy listę, a później traktujemy ją w standardowy sposób, tak jak List.fold_left. Jest też inny sposób, wykorzystujący kontynuacje, ale to jest daleko poza zakresem tego przedmiotu. <pre>let f g xs = let xs = List.rev xs in let rec aux xs acc = (match xs with [] -> acc x :: xs -> aux xs ((g x) :: acc)) in aux xs [g 0]</pre>

4.	<pre>let rec app xs ys = match xs with [] -> ys x :: xs -> x :: (app xs ys)</pre>	Z tych samych względów co wyżej, musimy użyć najpierw <code>List.rev</code> <pre>let app xs ys = let xs = List.rev xs in let rec aux xs ys = (match xs with [] -> ys x :: xs -> aux xs (x :: ys)) in aux xs ys</pre>
5.	<pre>let rec comp f fil a = if fil a then a else 1 + comp f fil (f a)</pre>	<pre>let comp f fil a = let rec aux a acc = if fil a then acc + a else aux (f a) (1 + acc) in aux a 0</pre>
6.	<pre>let rec f g xs acc = match xs with [] -> acc x :: xs -> f g xs (g acc x)</pre>	Tak. To jest <code>List.fold_left</code>, tylko z inną kolejnością argumentów.

V. Identyfikacja funkcji

W każdym rzędzie podana jest funkcja w postaci typu, definicji i jej działania. Wypełnij brakujące informacje.

Pierwszy wpis (dla `square`) jest podany jako przykład.

Lp	Typ funkcji	Definicja funkcji	Opis funkcji
1.	<code>int -> int</code>	<code>fun a -> a * a</code>	Podnosi argument <i>a</i> do kwadratu.
2.	<code>int -> 'a -> 'a list -> 'a list</code>		Wstawia <i>n</i> elementów <i>a</i> na początek listy <i>xs</i> .
3.		<code>fun a b c -> if a > 0 then b a else c a</code>	
4.	<code>(ident * expr) list -> ident -> expr -> (ident * expr) list</code>		Aktualizuje (jeżeli para z pierwszą współrzędną równą <i>ident</i> jest już w liście, to ją najpierw usuwa) listę par wprowadzając nową parę (<i>ident</i> , <i>expr</i>).
5.			Zamienia listę elementów typu τ opt na listę elementów typu τ , pozbywając się pustych elementów.

Rozwiązania

Lp	Typ funkcji	Definicja funkcji	Opis funkcji
1.	<code>int -> int</code>	<code>fun a -> a * a</code>	Podnosi argument a do kwadratu.
2.	<code>int -> 'a -> 'a list -> 'a list</code>	<pre>let rec cons n x xs = if n = 0 then xs else cons (n-1) x (x :: xs)</pre>	Wstawia n elementów a na początek listy xs .
3.	<code>int -> (int -> 'a) -> (int -> 'a) -> 'a</code>	<code>fun a b c -> if a > 0 then b a else c a</code>	$\text{fun } a \ b \ c = \begin{cases} b \ a, & \text{jeśli } a > 0 \\ c \ a, & \text{w.p.p} \end{cases}$
4.	<code>(ident * expr) list -> ident -> expr -> (ident * expr) list</code>	Zakładamy, że klucz w pierwszej współrzędnej pary może wystąpić tylko raz w liście. <pre>let update xs k v = let rec aux xs k v = (match xs with [] -> [] (k',v') :: xs -> if k' = k then (k,v) :: xs else (k',v') :: aux) xs k v in aux xs k v</pre>	Aktualizuje (jeżeli para z pierwszą współrzedną równą ident jest już w liście, to ją najpierw usuwa) listę par wprowadzając nową parę $(\text{ident}, \text{expr})$.
5.	<code>'a option list -> 'a list</code>	<pre>let rec unwind xs = match xs with [] -> [] Some x :: xs -> x :: unwind xs None :: xs -> unwind xs</pre>	Zamienia listę elementów typu τ opt na listę elementów typu τ , pozbywając się pustych elementów.

VI. Funkcje na listach

Zaimplementuj i otypuj poniższe funkcje biblioteczne. **Nie** możesz korzystać z funkcji bibliotecznych innych niż zaimplementowane.

Podkreśl nazwy funkcji które są rekurencyjne ogonowo.

Lp	Nazwa funkcji	Typ funkcji	Definicja funkcji
1.	<code>List.fold_left</code>		
2.	<code>List.fold_right</code>		
3.	<code>List.map</code>		
4.	<code>List.filter</code>		

Zaimplementuj funkcje opisane słownie poniżej, korzystając wyłącznie z powyżej wymienionych funkcji. (Jeżeli nie zaimplementowałeś danej funkcji, ale wiesz co ona robi, nadal możesz z niej skorzystać – poprawność przekazywania argumentów nie będzie wtedy brana pod uwagę)

W szczególności nie możesz definiować kolejnych funkcji rekurencyjnych.

Lp	Opis słowny	Definicja funkcji
1.	Funkcja przyjmująca listę liczb całkowitych, licząca ich sumę.	

2.	Funkcja przyjmująca listę list liczb całkowitych, licząca maksimum sum podlist.	
3.	Funkcja przyjmująca listę list zwracająca <code>true</code> wtw. gdy długości tych podlist są w kolejności rosnącej. (np. [(4);(5);(100)], gdzie (x) oznacza listę o długości x zwróci <code>true</code> , a [(3);(5);(4)] zwróci <code>false</code>)	
4.	Funkcja przyjmująca listę par typu (('a -> 'b) * 'a list), zwracająca listę list w których każdy element został zaaplikowany do odpowiedniej funkcji w pierwszej współrzędnej pary. (np. wynikiem [(fun x -> x * x), 2;3;4]) będzie [[4;9;16]])	

Rozwiązania

Lp	Nazwa funkcji	Typ funkcji	Definicja funkcji
1.	<u>List.fold_left</u>	('acc -> 'a -> 'acc) -> 'acc -> 'a list -> 'acc	<pre>let rec fold_left f acc xs = match xs with [] -> acc x :: xs -> fold_left f (f acc x) xs</pre>
2.	List.fold_right	('a -> 'acc -> 'acc) -> 'a list -> 'acc -> 'acc	<pre>let rec fold_right f xs acc = match xs with [] -> acc x :: xs -> f x (fold_right f xs acc)</pre>
3.	List.map	('a -> 'b) -> 'a list -> 'b list	<pre>let rec map f xs = match xs with [] -> [] x :: xs -> (f x) :: map f xs</pre>
4.	List.filter	('a -> bool) -> 'a list -> 'a list	<pre>let rec filter f xs = match xs with [] -> [] x :: xs -> if f x then x :: filter f xs else filter f xs</pre>

Lp	Opis słowny	Definicja funkcji
1.	Funkcja przyjmująca listę liczb całkowitych, licząca ich sumę.	<pre>let sum xs = fold_left (fun acc x -> x + acc) 0 xs</pre>
2.	Funkcja przyjmująca listę list liczb całkowitych, licząca maksimum sum podlist.	<pre>let max xss = fold_left (fun acc xs -> let sum = (fold_left (fun acc x -> acc + x) 0 xs) in if sum > acc then sum else acc) 0 xss</pre>
3.	Funkcja przyjmująca listę list zwracająca true wtw. gdy długości tych podlist są w kolejności rosnącej. (np. [(4);(5);(100)], gdzie (x) oznacza listę o długości x zwróci true, a [(3);(5);(4)] zwróci false)	<pre>let incr_lengths xss = let lengths = map (fun xs -> fold_left (fun acc x -> acc + 1) 0 xs) xss in (fold_left (fun acc x -> if acc = -2 then -2 else if x > acc then x else -2) (-1) lengths) <> -2</pre>
4.	Funkcja przyjmująca listę par typu (('a -> 'b) * 'a list), zwracająca listę list w których każdy element został	<pre>let map_map xss = map (fun (f, xs) -></pre>

	zaaplikowany do odpowiedniej funkcji w pierwszej współrzędnej pary. (np. wynikiem <code>[(fun x -> x * x), 2;3;4]</code> będzie <code>[[4;9;16]]</code>)	<pre>map f xs) xss</pre>
--	--	---------------------------

VII. Gramatyki

Dla poniżej zapisanych gramatyk określ czy są one jednoznaczne. Jeżeli nie, napisz kontrprzykład w pierwszej ramce. Tam gdzie wpiszesz kontrprzykład, zapisz zbiór produkcji aby gramatyka była równoważna. **Nie możesz** edytować zbiorów nieterminali, terminali, nieterminalu startowego.

1. $G = (N, T, P, S)$, gdzie:

$$N = \{S\},$$

$$T = \{a\},$$

$$P = \{$$

$$S \rightarrow aS,$$

$$S \rightarrow \varepsilon S,$$

$$S \rightarrow a$$

$$\}$$

$$P = \{$$

$$\}$$

2.

$G = (N, T, P, S)$, gdzie:

$$N = \{S\},$$

$$T = \{ (,) \},$$

$$P = \{$$

$$S \rightarrow SS,$$

$$S \rightarrow (S),$$

$$S \rightarrow \varepsilon$$

$$\}$$

$P = \{$

$\}$

3.

$G = (N, T, P, S)$, gdzie:

$$N = \{S, D\},$$

$$T = \{1, 2, 3, +, -\},$$

$$P = \{$$

$$D \rightarrow 1, D \rightarrow 2, D \rightarrow 3,$$

$$S \rightarrow D,$$

$$S \rightarrow S + S,$$

$$S \rightarrow S - S$$

$$\}$$

$P = \{$

$\}$

4.

$$\begin{aligned} G &= (N, T, P, S), \quad \text{gdzie:} \\ N &= \{S, V\}, \\ T &= \{\top, \perp, \Rightarrow, \neg, a, b, \dots, z\}, \\ P &= \{ \\ &\quad V \rightarrow \perp, V \rightarrow \top, \\ &\quad V \rightarrow a, V \rightarrow b, \dots, V \rightarrow z, \\ &\quad S \rightarrow V, \\ &\quad S \rightarrow S \Rightarrow S, \\ &\quad S \rightarrow \neg S \\ &\quad \} \end{aligned}$$

$P = \{$

$\}$

Wskazówki

- Staraj się znaleźć produkcje które generują ten sam nieterminal w izolacji, np dla

$$P = \{ \\ V \rightarrow \varepsilon \\ S \rightarrow V \\ S \rightarrow \varepsilon \\ \}$$

banalnym źródłem niejednoznaczności będzie wyprowadzenie ε na dwa sposoby:

1. $S \Rightarrow V \Rightarrow \varepsilon$,
2. $S \Rightarrow \varepsilon$

- Aby zbudować jednoznaczną gramatykę, najpierw uporządkuj produkcje dla tego samego nieterminala po ilości rekurencji.

Np. dla

$$P = \{ \\ S \rightarrow S \Rightarrow S, \\ S \rightarrow \neg S \\ S \rightarrow V \\ \}$$

oddziel od siebie $S \rightarrow S \Rightarrow S$ oraz $S \rightarrow \neg S$.

Następnie, posortuj te grupy rosnąco (po ilości rekurencyjnych wystąpień):

$$P = \{ \\ S \rightarrow \neg S \\ S \rightarrow S \Rightarrow S, \\ S \rightarrow V \\ \}$$

1. Dla każdej grupy musisz rozdzielić jaki nieterminal ją generuje.

$$P = \{ \\ V \rightarrow \neg S \\ S \rightarrow S \Rightarrow S, \\ S \rightarrow V \\ \}$$

2. Po zmianie nieterminala-źródła, zmień wszystkie jego wystąpienia w środku produkcji na ten nieterminal na który zmieniłeś.

$$P = \{ \\ V \rightarrow \neg V \\ S \rightarrow S \Rightarrow S, \\ S \rightarrow V \\ \}$$

3. (niemal) Każda produkcja która odwołuje się do siebie rekurencyjnie więcej niż raz (n razy), jest źródłem niejednoznaczności. W tych miejscach wymień $n - 1$ nieterminali na takie o wyższym priorytecie (wyższy priorytet == bliżej terminalów).

$$P = \{ \\ V \rightarrow \neg V \\ S \rightarrow V \Rightarrow S, \\ S \rightarrow V \\ \}$$

Najbardziej prawidłowo jest to robić według kierunku wiązania jakiegoś operatora, np. implikacja wiąże w prawo – po prawej stronie zostawiam więc rekurencyjne zawołanie do produkcji. Dla operatorów wiążących w lewo, np. dodawanie – zostawia się rekurencyjne wołanie po lewej stronie.

4. Gotowe – ciesz się swoją jednoznacznie zdefiniowaną gramatyką! Jeżeli nadal nie jest jednoznaczna, trzeba pewnie powtórzyć procedurę, lub ją lekko zmodyfikować, w najgorszym przypadku dokładając kolejne stopnie priorytetów pomiędzy terminalami a nieterminalem startowym.

Pamiętaj, że (poza szczególnymi case'ami takimi jak (`expr`)) zawołania innych produkcji powinny być o priorytecie **wyższym** niż źródłowa.

Odpowiedzi

1.

$G = (N, T, P, S)$, gdzie:

$$N = \{S\},$$

$$T = \{a\},$$

$$P = \{$$

$$S \rightarrow aS,$$

$$S \rightarrow \varepsilon S,$$

$$S \rightarrow a$$

}



$$P = \{$$
$$S \rightarrow aS,$$
$$S \rightarrow \varepsilon$$
$$\}$$

2.

$G = (N, T, P, S)$, gdzie:

$$N = \{S\},$$

$$T = \{ (,) \},$$

$$P = \{$$

$$S \rightarrow SS,$$

$$S \rightarrow (S),$$

$$S \rightarrow \varepsilon$$

$\}$

S
|
 ε

S
|
^
 S S
| |
 ε ε

$P = \{$

$$S \rightarrow (S)S,$$

$$S \rightarrow \varepsilon$$

$\}$

3.

$G = (N, T, P, S)$, gdzie:

$N = \{S, D\}$,

$T = \{1, 2, 3, +, -\}$,

$P = \{$

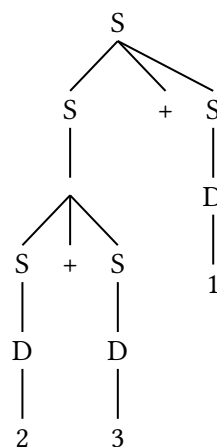
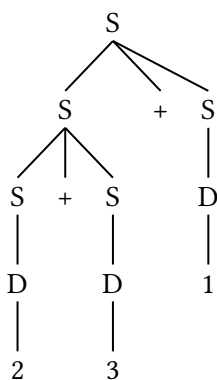
$D \rightarrow 1, D \rightarrow 2, D \rightarrow 3,$

$S \rightarrow D,$

$S \rightarrow S + S,$

$S \rightarrow S - S$

$\}$



$P = \{$

$D \rightarrow 1, D \rightarrow 2, D \rightarrow 3,$

$S \rightarrow D,$

$S \rightarrow S + D,$

$S \rightarrow S - D$

$\}$

4.

$G = (N, T, P, S)$, gdzie:

$N = \{S, V\}$,

$T = \{\top, \perp, \Rightarrow, \neg, a, b, \dots, z\}$,

$P = \{$

$V \rightarrow \perp, V \rightarrow \top,$

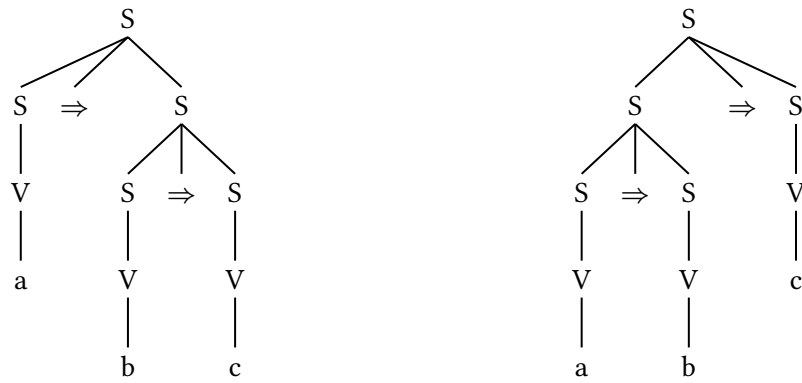
$V \rightarrow a, V \rightarrow b, \dots, V \rightarrow z,$

$S \rightarrow V,$

$S \rightarrow S \Rightarrow S,$

$S \rightarrow \neg S$

$\}$



$P = \{$

$V \rightarrow \neg V$

$V \rightarrow \perp, V \rightarrow \top,$

$V \rightarrow a, V \rightarrow b, \dots, V \rightarrow z,$

$S \rightarrow V,$

$S \rightarrow V \Rightarrow S,$

$\}$

VIII. Abstrakcyjne typy danych

VIII.1 Rozwinięcie typów języka

VIII.1.1 Język arytmetycznych wyrażeń

Rozważamy język wyrażeń arytmetycznych na liczbach całkowitych z dodawaniem, odejmowaniem, dzieleniem, mnożeniem. Rozwiń następujące typy:

`type binop =`

`type expr =`

VIII.1.2 Język *calc_let*

Rozważamy język gdzie wartości mogą wartościami algebry Boole'a, liczbami całkowitymi, parami i listami wartości. Napisz typ dla *value*:

`type value =`

Dla liczb całkowitych definiujemy operację dodawania, odejmowania, mnożenia, dzielenia. Dla takich samych typów definiujemy relacje $<$, $>$, $\geq$, $\leq$, $=$. Poza tym definiujemy następujące wyrażenia:

- związanie wyrażenia do zmiennej w wyrażeniu: `let x = e1 in e2`,
- operator `cons` do dostawienia wartości na początku listy,
- `fst` i `snd` dla list.

Dany jest następujący typ składni abstrakcyjnej:

`type bop = Mult | Div | Add | Sub`

`type var = string`

```
type expr =  
  | Var    of var  
  | Int    of int  
  | Binop  of bop * expr * expr  
  | Let    of var * expr * expr  
  | Pair   of expr * expr  
  | Fst    of expr  
  | Snd    of expr
```

Dane są następujące tokeny:

```
IDENT  
MULT DIV ADD SUB EQ  
LPAR RPAR COMMA  
FST SND  
LET IN  
EOF
```

Uzupełnij brakującą definicję gramatyki w *menhir*:

`expr:`

`| LET;`

`| e = opexpr { e }`

`;`

`opexpr:`

`| i = INT { Int i }`

`| LPAR; e = expr; RPAR { e }`

`| LPAR; e1 = expr; COMMA; e2 = expr; RPAR { Pair(e1, e2) }`

`| SND; e = expr; { Snd(e) }`

`| x = IDENT { Var x }`

`;`

VIII.2 Monady

Informacja

Monady w tak bezpośredni sposób jak poniżej nie pojawiły się na wykładzie ani ćwiczeniach, ale takie pytanie ma szansę pojawić się na egzaminie.

Czym jest monada? Najprościej wyjaśnić monadę jako sposób pisania kodu w którym mamy jakiś abstrakcyjny typ `'a monada`, funkcję `return` która zamienia typ `'a` na `'a monada`, która podnosi wartość do kontekstu monady, oraz funkcję `bind` typu `'a monada -> ('a -> 'b monada) -> 'b monada`, która wyciąga wartość z kontekstu monady i aplikuje ją na funkcję.

[Wikipedia](#)

Rozważmy monadę typu `option`, zdefiniowaną następująco:

```
type 'a option =  
  | Some of 'a  
  | None
```

Napisz funkcje `return` i `bind`.

```
let return v =
```

```
let bind m f =
```


IX. Indukcja strukturalna

IX.1 Formułowanie zasady indukcji

IX.1.1 Zadanie A.

Rozważmy następujący typ danych reprezentujący wyrażenia złożone z zer, operacji następnika i dodawań.

```
type expr =  
  | Zero  
  | Succ of expr  
  | Add of expr * expr
```

Sformułuj zasadę indukcji dla tego typu danych.

Rozwiązanie

```
type expr =  
  | Zero  
  | Succ of expr  
  | Add of expr * expr
```

Niech P będzie predykatem dotyczącym wartości typu `expr`. Jeżeli zachodzą następujące warunki:

- $P(\text{Zero})$,
- dla każdego e typu `expr` jeśli $P(e)$ to $P(\text{Succ}(e))$,
- dla każdego e_1, e_2 typu `expr` jeśli $P(e_1)$ oraz $P(e_2)$, to $P(\text{Add}(e_1, e_2))$,

to dla każdego $e : \text{expr}$ zachodzi predykat $P(e)$.

Skrócony zapis:

Niech $P : \text{expr} \rightarrow \text{bool}$:

- $P(\text{Zero})$,
- $\forall e : \text{expr}. P(e) \Rightarrow P(\text{Succ}(e))$,
- $\forall e_1, e_2 : \text{expr}. P(e_1) \wedge P(e_2) \Rightarrow P(\text{Add}(e_1, e_2))$,

, wtedy $\forall e : \text{expr}. P(e)$

IX.1.2 Zadanie B.

Definiujemy następujący typ dla drzew:

```
type 'a tree =  
  | Leaf  
  | Node of 'a tree * 'a * 'a tree
```

Sformułuj zasadę indukcji dla tego typu danych.

Dla tego typu danych definiujemy dwa warianty funkcji `flatten`, spłaszczającej drzewo do listy w kolejności (wynikiem będzie posortowana lista):

```
let rec flatten1 t = match t with  
  | Leaf -> []  
  | Node(t1, x, t2) -> flatten1 t1 @ x :: flatten1 t2  
  
let flatten2 t =  
  let rec flatten2_aux t xs = match t with  
    | Node(t1, x, t2) -> flatten2_aux t1 (x :: flatten2_aux t2 xs)  
    | Leaf -> xs  
  in flatten2_aux t []
```

Udowodnij indukcyjnie, że dla każdego drzewa t : `'a tree` zachodzi własność $\text{flatten1 } t \equiv \text{flatten2 } t$. Aby to zrobić, najpierw sformułuj i udowodnij lemat na temat funkcji `flatten1` i `flatten2_aux`.